

Chapter: SQL Advanced Aggregations

GROUPING SETS is the most flexible way to generate summaries in SQL.

Think of it as:

"I don't want all subtotals (like CUBE) or only hierarchical subtotals (like ROLLUP). I want to explicitly choose which summaries to generate."

It is the foundation upon which ROLLUP and CUBE are conceptually built.

Why GROUPING SETS Exists

Suppose your company wants a sales report.

Department	City	Sales
ECE	Delhi	500
ECE	Mumbai	300
CSE	Delhi	700
CSE	Mumbai	400

The manager asks for:

- Sales by Department
- Sales by City
- Sales by Department + City
- Grand Total

Nothing else.

ROLLUP or CUBE may produce summaries you don't need.

GROUPING SETS lets you request only these four reports.

14.19.2 Without GROUPING SETS

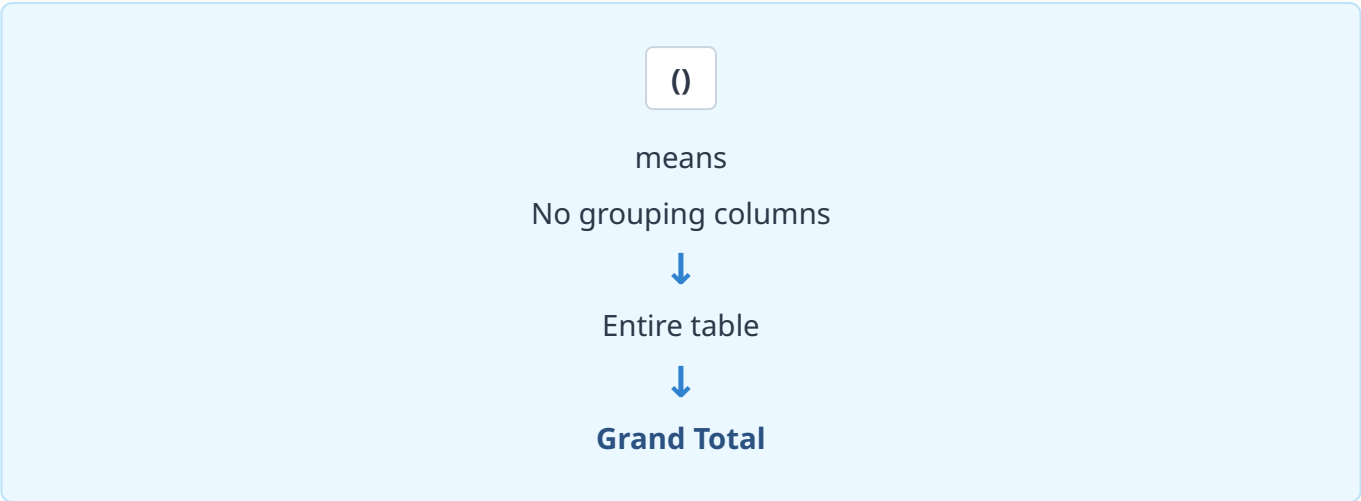
Normally you would write

```
SELECT department,
       NULL AS city,
       SUM(sales)
FROM Sales
GROUP BY department
UNION ALL
SELECT NULL,
       city,
       SUM(sales)
FROM Sales
GROUP BY city
UNION ALL
SELECT department,
       city,
       SUM(sales)
FROM Sales
GROUP BY department,city
UNION ALL
SELECT NULL,
       NULL,
       SUM(sales)
FROM Sales;
```

GROUPING SETS Syntax

```
SELECT
department,
city,
SUM(sales)
FROM Sales
GROUP BY
GROUPING SETS
(
(department,city),
(department),
(city),
()
);
```

Meaning of ()



Output

Department	City	Sales
ECE	Delhi	500
ECE	Mumbai	300
CSE	Delhi	700
CSE	Mumbai	400
ECE	NULL	800
CSE	NULL	1100
NULL	Delhi	1200
NULL	Mumbai	700
NULL	NULL	1900

GROUPING SETS vs ROLLUP

ROLLUP

```
GROUP BY ROLLUP(department,city)
```

Automatically generates

Department + City



Department



Grand Total

You cannot skip one.

GROUPING SETS

```
GROUPING SETS  
(  
  (department),  
  (city)  
)
```

Generates only

- Department
- City

Nothing else.

GROUP BY vs ROLLUP vs CUBE vs GROUPING SETS

Feature	GROUP BY	ROLLUP	CUBE	GROUPING SETS
Detailed groups	✓	✓	✓	✓
Hierarchy totals	✗	✓	✓	If specified
Every subtotal	✗	✗	✓	If specified
Custom subtotal selection	✗	✗	✗	✓
Grand total	✗	✓	✓	If specified

When using ROLLUP, CUBE, or GROUPING SETS, SQL generates rows containing NULL.

But here's the problem:

Did the original data contain NULL, or did SQL create it as a subtotal?

GROUPING() answers that question.

14.20.1 The Problem

Suppose

Department	City	Sales
ECE	Delhi	500
ECE	NULL	300

This NULL is real data.

Now run

```
GROUP BY ROLLUP(department,city)
```

Output

Department	City
ECE	Delhi
ECE	NULL
ECE	NULL

They look identical.

14.20.2 Syntax

```
GROUPING(column)
```

Returns

Value	Meaning
0	Original column value (including real NULLs)
1	Column omitted because this is a subtotal/grand-total row

14.20.3 Example

```
SELECT
department,
city,
SUM(sales),
GROUPING(city)
FROM Sales
GROUP BY ROLLUP(department,city);
```

Output

Department	City	GROUPING(city)
ECE	Delhi	0
ECE	Mumbai	0
ECE	NULL	1

The subtotal row is easy to identify.

14.20.4 Multiple Columns

```
SELECT
department,
city,
GROUPING(department),
GROUPING(city)
FROM Sales
GROUP BY ROLLUP(department,city);
```

Possible output

Department	City	Dept	City
ECE	Delhi	0	0
ECE	NULL	0	1
NULL	NULL	1	1

op-selling Category

```
SELECT
category,
SUM(amount) AS revenue
FROM Orders
GROUP BY category
ORDER BY revenue DESC
LIMIT 1;
```

2. Revenue by Month

```
SELECT
YEAR(order_date) AS year,
MONTH(order_date) AS month,
SUM(amount) AS revenue
FROM Orders
GROUP BY
YEAR(order_date),
MONTH(order_date)
ORDER BY
year,
month;
```

3. Customers with More Than 10 Orders

```
SELECT
customer_id,
COUNT(*) AS total_orders
FROM Orders
GROUP BY customer_id
HAVING COUNT(*) > 10;
```

4. Employees Earning Above Department Average

```
SELECT *
FROM Employees e
WHERE salary >
(SELECT AVG(salary)
FROM Employees
WHERE department = e.department
);
```

This is a correlated subquery and is a classic interview problem.

5. Store-wise Yearly Sales

```
SELECT
store_id,
YEAR(order_date) AS year,
SUM(amount) AS revenue
FROM Orders
GROUP BY
store_id,
YEAR(order_date);
```